Agentic RAG Chatbot Architecture

Using Model Context Protocol (MCP)

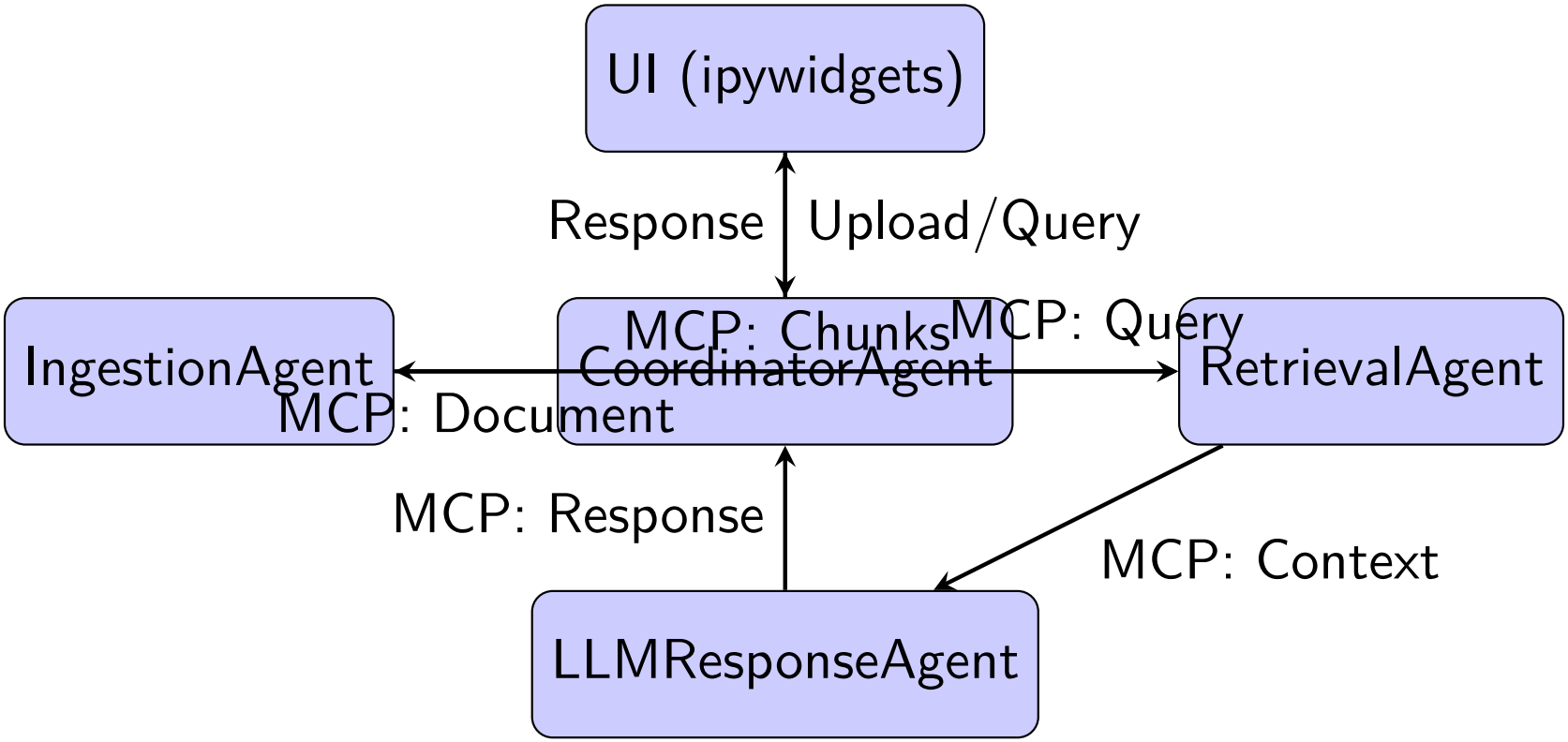
kunal Saurav

July 25, 2025

Agent-Based Architecture

- **IngestionAgent**: Parses PDF, DOCX, PPTX, CSV, TXT/Markdown; splits into chunks.
- **RetrievalAgent**: Embeds chunks using HuggingFace model; stores in ChromaDB; retrieves relevant chunks.
- **LLMResponseAgent**: Formats query with context; generates response (mocked).
- **CoordinatorAgent**: Orchestrates message passing via MCP.

System Flow Diagram



Tech Stack

- **Backend**: Python, LangChain, ChromaDB, HuggingFace Embeddings
- **Frontend**: Jupyter Notebook with ipywidgets
- **Document Parsing**: PyPDF2, python-docx, python-pptx, pandas, markdown
- **MCP**: In-memory JSON-based messaging

UI Screenshots

- File Upload Widget: Supports PDF, DOCX, PPTX, CSV, TXT, Markdown
- Query Input: Text box for multi-turn questions
- Output Area: Displays responses and source chunks

Challenges Faced

- **Document Parsing**: Handling inconsistent formats and encodings.
- **Embedding Scalability**: Managing memory for large documents.
- **MCP Implementation**: Ensuring robust message passing without external dependencies.
- **UI in Jupyter**: Limited interactivity compared to web frameworks.

Future Scope

- Integrate real LLM (e.g., via xAI API: `https://x.ai/api`).
- Support real-time document streaming.
- Enhance UI with React or Streamlit for production.
- Add multi-language support for documents.